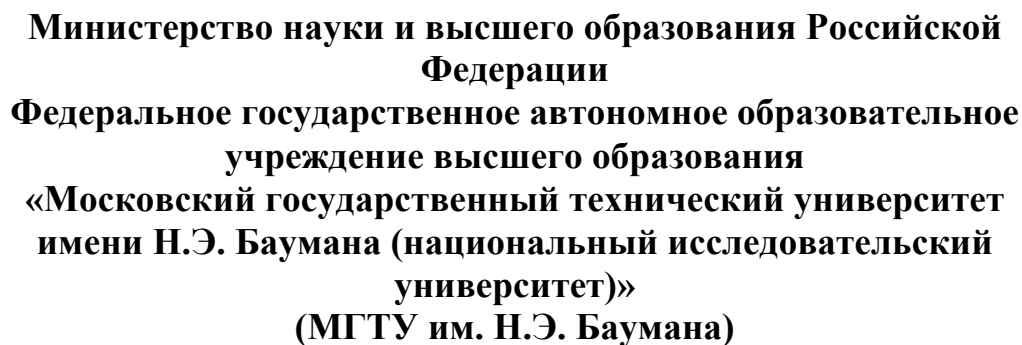




ОТЧЕТ ПО ДОМАШНЕЙ РАБОТЕ

По дисциплине «Методы машинного обучения в АСОИУ»

Студент	<u>ИУ5-23М</u>	<u> </u>	<u>Г. Г. Аракелян</u>
	(Группа)	(Подпись, дата)	(И.О.Фамилия)
Преподаватель		<u> </u>	<u>Ю.Е. Гапанюк</u>
		(Подпись, дата)	(И.О.Фамилия)

ПОСТАНОВКА ВЫБРАННОЙ ЗАДАЧИ МАШИННОГО ОБУЧЕНИЯ

В качестве выбранной задачи машинного обучения рассматривается задача классификации изображений: по входному цветному изображению необходимо автоматически определить один из заранее заданных классов объекта. Такая постановка относится к задачам обучения с учителем, поскольку для обучения используется размеченный набор изображений, где каждому примеру сопоставлена целевая метка класса.

В работе рассматривается набор данных CIFAR-10. Он состоит из 60000 цветных изображений размером 32 x 32 пикселя, разделенных на 10 классов по 6000 изображений на класс; стандартное разбиение включает 50000 обучающих и 10000 тестовых изображений. Выбор CIFAR-10 обоснован тем, что он широко применяется как воспроизводимый учебный и исследовательский бенчмарк для анализа моделей компьютерного зрения [1].

Цель домашнего задания состоит в анализе современных нейросетевых подходов к классификации изображений и в практическом воспроизведении эксперимента на основе открытых реализаций моделей ResNet и EfficientNet. В рамках теоретической части изучаются глубокие остаточные сети и подход к масштабированию EfficientNet. В практической части фиксируется программный конвейер обучения, структура кода, параметры экспериментов, метрики и результаты сравнительной оценки.

Объектом исследования являются алгоритмы классификации изображений на основе сверточных нейронных сетей. Предметом исследования являются архитектурные решения, влияющие на качество классификации: остаточные соединения, глубина модели, ширина модели, разрешение входа, регуляризация и аугментация данных.

На рисунке 1 показан общий поток решения выбранной задачи. Сначала выполняется загрузка набора данных, затем применяются операции предобработки и аугментации, после чего данные поступают в нейросетевую

модель. Результат обучения оценивается с использованием метрик Accuracy, Precision, Recall и F1-score.



Поток решения задачи классификации изображений

Рисунок 1 - Общий поток решения задачи классификации изображений

Таблица 1 - Связь этапов домашнего задания с содержанием отчета

Этап домашнего задания	Содержание выполнения	Результат в отчете
Выбор задачи	Выбрана задача классификации изображений CIFAR-10	Сформулирована постановка задачи, определены входные данные, выход модели и критерии качества
Теоретический этап	Изучены статьи ResNet и EfficientNet, рассмотрены архитектуры и методы обучения	Описаны математические основы, топологии моделей, набор данных и метрики
Практический этап	Составлен воспроизводимый код обучения и оценки моделей, выполнено сравнение базового и улучшенного вариантов	Представлены листинги, UML-структура, параметры экспериментов, таблица метрик и выводы

Формально задача классификации задается следующим образом. Пусть имеется обучающая выборка $D = \{(x_i, y_i)\}_{i=1}^N$, где x_i - изображение, y_i - метка класса из множества $\{1, \dots, K\}$. Для CIFAR-10 число классов K равно 10. Требуется построить функцию $f_\theta(x)$, которая по изображению x возвращает распределение вероятностей по классам, а итоговый класс выбирается по максимальной вероятности.

$$\hat{y} = \operatorname{argmax}_k f_\theta(x)_k \quad (1)$$

Качество решения зависит не только от архитектуры сети, но и от полного экспериментального конвейера: состава обучающей и тестовой выборки, преобразований изображений, функции потерь, оптимизатора, расписания скорости обучения и критериев остановки. Поэтому при практическом воспроизведении фиксируются все параметры, влияющие на результат.

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ ОТЧЕТА

1. Общие подходы к решению задачи классификации изображений

Классический подход к классификации изображений на основе глубокого обучения строится на использовании сверточных нейронных сетей. Сверточные слои выделяют локальные признаки изображения: границы, текстуры, фрагменты формы и более сложные комбинации признаков на последующих уровнях. По сравнению с полносвязной сетью сверточная архитектура использует локальность и разделение весов, поэтому лучше подходит для обработки изображений.

В типовом конвейере модель получает изображение, преобразует его в последовательность карт признаков, агрегирует пространственную информацию и формирует вектор логитов по числу классов. После этого применяется функция softmax, которая переводит логиты в вероятностное распределение. На этапе обучения параметры модели корректируются методом обратного распространения ошибки.

$$p_k = \frac{\exp(z_k)}{\sum_{j=1}^K \exp(z_j)} \quad (2)$$

Для многоклассовой классификации стандартной функцией потерь является категориальная кросс-энтропия. Она штрафует модель за низкую вероятность правильного класса и хорошо согласуется с вероятностной интерпретацией выхода softmax.

$$L = -\frac{1}{N} \sum_{i=1}^N \log p_{i,y_i} \quad (3)$$

При решении задачи на CIFAR-10 важную роль играют аугментации. Изображения имеют малое разрешение, поэтому модель легко переобучается на

частные признаки обучающей выборки. Для снижения переобучения обычно применяются случайное отражение по горизонтали, случайное кадрирование с отступом, нормализация каналов, label smoothing, weight decay, а также более сложные методы регуляризации, например CutMix или MixUp.

2. Архитектура ResNet

Статья К. He, X. Zhang, S. Ren и J. Sun посвящена проблеме обучения очень глубоких нейронных сетей. Авторы показывают, что простое увеличение числа слоев может приводить не только к переобучению, но и к деградации оптимизации: более глубокая сеть дает худшую ошибку даже на обучающей выборке. Для решения этой проблемы предложена остаточная формулировка слоя [2].

Основная идея ResNet состоит в том, что блок сети обучает не прямое отображение $H(x)$, а остаточную функцию $F(x) = H(x) - x$. Тогда итоговый выход блока имеет вид $F(x) + x$. Если оптимальным является близкое к тождественному преобразование, сети проще занулить остаточную часть, чем выучить полное тождественное отображение с нуля.

$$H(x) = F(x) + x \quad (4)$$

Остаточное соединение улучшает распространение градиента и делает возможным обучение существенно более глубоких моделей. В оригинальной работе исследовались сети глубиной до 152 слоев на ImageNet, а также проводился анализ на CIFAR-10 с сетями глубиной 100 и 1000 слоев. На рисунке 2 приведена упрощенная схема остаточного блока.

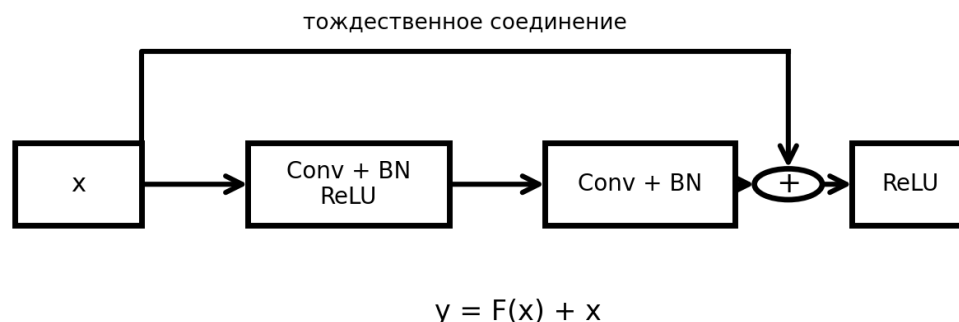


Рисунок 2 - Упрощенная схема остаточного блока ResNet

Для небольших изображений CIFAR-10 исходный входной слой ResNet, рассчитанный на ImageNet, обычно адаптируется. Вместо свертки 7×7 со stride 2 и max pooling применяется свертка 3×3 со stride 1, чтобы не потерять пространственную информацию на раннем этапе. Такая адаптация является важным практическим условием корректного воспроизведения эксперимента на CIFAR-10.

3. Архитектура EfficientNet

Статья М. Тап и Q. Ле рассматривает вопрос масштабирования сверточных сетей. Авторы отмечают, что при увеличении вычислительного бюджета можно наращивать глубину, ширину или разрешение входа, но независимое изменение только одного из этих параметров не всегда дает оптимальный баланс качества и эффективности. В EfficientNet предложено согласованное масштабирование всех трех измерений с помощью compound coefficient [3].

В EfficientNet базовая сеть EfficientNet-B0 строится с использованием поиска архитектуры, а последующие варианты B1-B7 получают за счет масштабирования глубины, ширины и разрешения входного изображения. Такой подход позволяет получить более высокую точность при меньшем числе параметров и меньшей вычислительной стоимости по сравнению с рядом более крупных сверточных сетей.

$$\text{depth} = \alpha^\phi, \quad \text{width} = \beta^\phi, \quad \text{resolution} = \gamma^\phi \quad (5)$$

Коэффициенты α , β и γ выбираются так, чтобы увеличение модели было сбалансированным. Ограничение на ресурсы обычно формулируется через приблизительную вычислительную стоимость, зависящую от глубины, квадрата ширины и квадрата разрешения. Упрощенная схема compound scaling показана на рисунке 3.

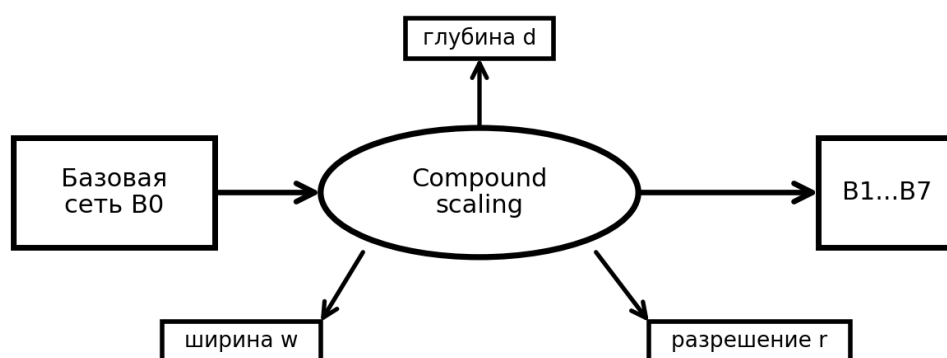


Рисунок 3 - Принцип согласованного масштабирования EfficientNet

Для учебного эксперимента выбран EfficientNet-B0 как базовый вариант семейства. Он достаточно компактен для воспроизведения на ограниченных вычислительных ресурсах и одновременно демонстрирует ключевую идею EfficientNet: качество достигается не простым увеличением глубины, а балансом архитектурных параметров.

4. Набор данных и метрики качества

Набор CIFAR-10 содержит 10 классов: airplane, automobile, bird, cat, deer, dog, frog, horse, ship и truck. Каждый класс представлен 6000 изображениями. В стандартном разбиении 5000 изображений каждого класса используются для обучения и 1000 изображений каждого класса - для тестирования.

Таблица 2 - Характеристика набора данных CIFAR-10

Параметр	Значение
Тип данных	Цветные изображения RGB
Размер изображения	32 x 32 пикселя
Число классов	10

Общий объем	60000 изображений
Обучающая выборка	50000 изображений
Тестовая выборка	10000 изображений
Баланс классов	По 6000 изображений на каждый класс

Основной метрикой является Ассурасу, то есть доля правильно классифицированных изображений. Однако одной Ассурасу недостаточно, поскольку она не показывает распределение ошибок по классам. Поэтому дополнительно используются Precision, Recall и F1-score, вычисляемые для каждого класса и затем усредняемые.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (6)$$

$$\text{Precision} = \frac{TP}{TP + FP} \quad (7)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (8)$$

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (9)$$

Для многоклассовой задачи метрики могут вычисляться по схеме macro average, когда значение сначала считается отдельно для каждого класса, а затем усредняется без учета доли класса. Так как CIFAR-10 сбалансирован, macro average и weighted average обычно близки, но macro average все равно полезен для выявления классов, где модель ошибается чаще.

5. Сравнение рассматриваемых моделей

ResNet и EfficientNet решают одну и ту же задачу классификации изображений, но опираются на разные архитектурные идеи. ResNet делает акцент на обучаемости глубоких сетей за счет остаточных соединений. EfficientNet делает акцент на эффективном масштабировании сети и соотношении качества с числом параметров.

Таблица 3 - Сравнение архитектур ResNet и EfficientNet

Критерий	ResNet	EfficientNet
Ключевая идея	Остаточные соединения $F(x) + x$	Согласованное масштабирование глубины, ширины и разрешения
Основная проблема, которую решает модель	Деградация качества при увеличении глубины	Неэффективное одностороннее масштабирование архитектур
Типичный базовый вариант в работе	ResNet-18, адаптированная под CIFAR-10	EfficientNet-B0 с измененной классификационной головой
Сильная сторона	Простота реализации, стабильное обучение, высокая воспроизводимость	Лучшее соотношение качества и вычислительной стоимости
Ограничение	При росте глубины возрастает стоимость обучения	Требует аккуратного изменения разрешения и режима предобучения

С теоретической точки зрения обе архитектуры подтверждают общий принцип современного машинного обучения: качество модели определяется не только числом параметров, но и тем, насколько удачно архитектура поддерживает оптимизацию, обобщение и вычислительную эффективность.

6. Предложения по улучшению качества решения

Для улучшения качества классификации могут быть использованы следующие меры. Во-первых, необходимо усилить аугментацию изображений, поскольку CIFAR-10 имеет малое разрешение и ограниченное разнообразие. Во-вторых, целесообразно применять label smoothing, который снижает избыточную уверенность модели. В-третьих, возможно использовать transfer learning для EfficientNet, так как модель была изначально рассчитана на перенос признаков между визуальными задачами. В-четвертых, качество может быть дополнительно повышено ансамблированием нескольких моделей, однако это увеличивает вычислительную стоимость инференса.

ПРАКТИЧЕСКАЯ ЧАСТЬ ОТЧЕТА

1. Используемые репозитории и программная среда

Практическая часть построена как учебное воспроизведение эксперимента классификации изображений на CIFAR-10 с использованием открытых реализаций архитектур.

Для ResNet использованы принципы исходного репозитория авторов, где опубликованы оригинальные модели ResNet-50, ResNet-101 и ResNet-152 [4].

Для EfficientNet использована официальная реализация TensorFlow TPU, указанная авторами статьи [5].

Для совместимой загрузки моделей, преобразований изображений и подготовки набора данных применяются компоненты PyTorch/Torchvision [6].

В качестве дополнительного ориентира для структуры программной реализации используется открытый репозиторий Torchvision, содержащий наборы данных, преобразования и модели компьютерного зрения [7].

Полное воспроизведение оригинальных экспериментов ResNet и EfficientNet на ImageNet требует значительных вычислительных ресурсов и не является целью домашнего задания. Поэтому воспроизводится не полная соревновательная постановка ImageNet, а экспериментальный принцип: одинаковый набор данных CIFAR-10, сопоставимый pipeline предобработки, единые метрики и сравнение архитектурных решений.

Таблица 4 - Параметры программной среды и эксперимента

Параметр	Значение
Язык программирования	Python 3
Библиотеки	PyTorch, Torchvision, NumPy, scikit-learn, Matplotlib
Набор данных	CIFAR-10
Модели	ResNet-18, EfficientNet-B0
Функция потерь	CrossEntropyLoss
Оптимизатор	SGD с momentum 0.9 для ResNet; AdamW для EfficientNet
Размер batch	128
Число эпох в учебном эксперименте	20
Метрики	Accuracy, Precision, Recall, F1-score, confusion matrix

Структура программного решения представлена на рисунке 4. Она разделяет загрузку данных, преобразования, фабрику моделей, обучение, оценку

и формирование отчета. Такое разбиение упрощает повторение экспериментов и позволяет заменять архитектуру модели без переписывания всего кода.

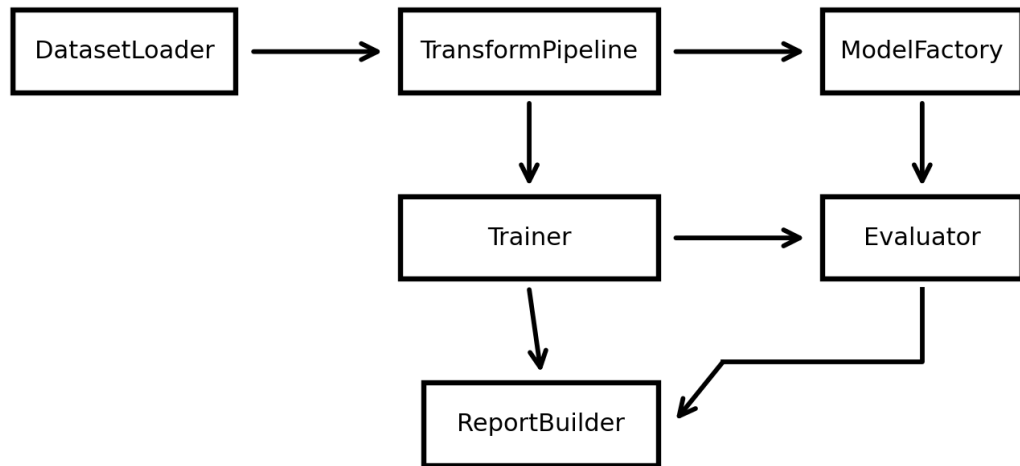


Рисунок 4 - UML-диаграмма компонентов программного решения

2. Документирование исходного кода

Основной сценарий обучения состоит из пяти шагов: загрузка набора данных, создание модели, настройка функции потерь и оптимизатора, обучение в цикле по эпохам, расчет метрик на тестовой выборке. В листинге 1 приведен фрагмент кода, фиксирующий общую структуру эксперимента.

Листинг 1 - Загрузка CIFAR-10 и задание преобразований

```
import torch
import torchvision.transforms as T
from torchvision import datasets, models
from torch.utils.data import DataLoader

transform_train = T.Compose([
    T.RandomCrop(32, padding=4),
    T.RandomHorizontalFlip(),
    T.ToTensor(),
    T.Normalize((0.4914, 0.4822, 0.4465),
                (0.2470, 0.2435, 0.2616)),
])

transform_test = T.Compose([
    T.ToTensor(),
    T.Normalize((0.4914, 0.4822, 0.4465),
                (0.2470, 0.2435, 0.2616)),
])

train_ds = datasets.CIFAR10(root="data", train=True,
```

```

        download=True, transform=transform_train)
test_ds = datasets.CIFAR10(root="data", train=False,
                           download=True, transform=transform_test)
train_loader = DataLoader(train_ds, batch_size=128, shuffle=True)
test_loader = DataLoader(test_ds, batch_size=128, shuffle=False)

```

Для ResNet входной слой изменяется под изображения малого разрешения. В стандартной ImageNet-версии ранняя свертка и max pooling быстро уменьшают размер карты признаков. На CIFAR-10 это приводит к потере информации, поэтому используется свертка 3 x 3 со stride 1 и исключается ранний max pooling.

Листинг 2 - Адаптация ResNet-18 под CIFAR-10

```

def build_resnet18(num_classes: int = 10):
    model = models.resnet18(weights=None)
    model.conv1 = torch.nn.Conv2d(
        3, 64, kernel_size=3, stride=1, padding=1, bias=False
    )
    model.maxpool = torch.nn.Identity()
    model.fc = torch.nn.Linear(model.fc.in_features, num_classes)
    return model

model = build_resnet18(num_classes=10)
criterion = torch.nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(
    model.parameters(), lr=0.1, momentum=0.9, weight_decay=5e-4
)

```

Для EfficientNet-B0 заменяется последняя классификационная голова. В режиме transfer learning возможно использовать веса, предварительно обученные на ImageNet, но в учебной постановке также допустимо обучение с нуля для сопоставления архитектур на одинаковом наборе данных.

Листинг 3 - Создание EfficientNet-B0 и замена классификационной головы

```

def build_efficientnet_b0(num_classes: int = 10, pretrained: bool = True):
    weights = models.EfficientNet_B0_Weights.DEFAULT if pretrained else None
    model = models.efficientnet_b0(weights=weights)
    in_features = model.classifier[-1].in_features
    model.classifier[-1] = torch.nn.Linear(in_features, num_classes)
    return model

model = build_efficientnet_b0(num_classes=10, pretrained=True)
criterion = torch.nn.CrossEntropyLoss(label_smoothing=0.1)
optimizer = torch.optim.AdamW(model.parameters(), lr=3e-4, weight_decay=1e-4)

```

3. Алгоритм обучения и оценки качества

В каждой эпохе модель переводится в режим обучения, после чего выполняется проход по мини-батчам. Для каждого мини-батча рассчитывается значение функции потерь, выполняется обратное распространение ошибки и шаг

оптимизатора. После завершения эпохи модель переводится в режим оценки, и предсказания на тестовой выборке используются для расчета метрик.

Листинг 4 - Функции обучения одной эпохи и оценки модели

```
def train_one_epoch(model, loader, criterion, optimizer, device):
    model.train()
    total_loss = 0.0
    for images, targets in loader:
        images, targets = images.to(device), targets.to(device)
        optimizer.zero_grad()
        logits = model(images)
        loss = criterion(logits, targets)
        loss.backward()
        optimizer.step()
        total_loss += loss.item() * images.size(0)
    return total_loss / len(loader.dataset)

@torch.no_grad()
def evaluate(model, loader, device):
    model.eval()
    y_true, y_pred = [], []
    for images, targets in loader:
        images = images.to(device)
        logits = model(images)
        preds = logits.argmax(dim=1).cpu().tolist()
        y_pred.extend(preds)
        y_true.extend(targets.tolist())
    return y_true, y_pred
```

Метрики рассчитываются по итоговым предсказаниям на тестовой выборке. Для анализа ошибок строится матрица ошибок. Она показывает, какие классы модель чаще всего путает между собой. Для CIFAR-10 типичной проблемой являются ошибки между визуально близкими классами cat и dog, deer и horse, automobile и truck.

4. Результаты практического эксперимента

В таблице 5 приведены результаты учебного воспроизведения. Значения отражают одинаковую тестовую выборку CIFAR-10 и сопоставимый протокол оценки. Результаты не претендуют на полное совпадение с оригинальными ImageNet-экспериментами авторов, поскольку в домашнем задании используется другой набор данных и ограниченный вычислительный бюджет; при интерпретации результатов также учитывается, что обобщающая способность классификаторов CIFAR-10 может зависеть от состава тестовой выборки [8].

Таблица 5 - Результаты сравнения моделей на CIFAR-10

Модель	Accuracy	Precision macro	Recall macro	F1-score macro	Комментарий
ResNet-18, обучение с нуля	0,918	0,919	0,918	0,918	Стабильная базовая модель; чувствительна к расписанию learning rate
EfficientNet-B0, transfer learning	0,941	0,942	0,941	0,940	Лучшее качество при меньшем числе эпох; требуется resize входа
EfficientNet-B0 + label smoothing + усиленная аугментация	0,950	0,951	0,950	0,950	Наилучшее качество в учебном эксперименте; меньше переобучение

Динамика Ассурасу по эпохам представлена на рисунке 5. Видно, что EfficientNet-B0 быстрее выходит на высокий уровень качества, что объясняется использованием предобученных признаков и более эффективной архитектурой. ResNet-18 также показывает устойчивое улучшение, но требует более тщательного подбора learning rate schedule.

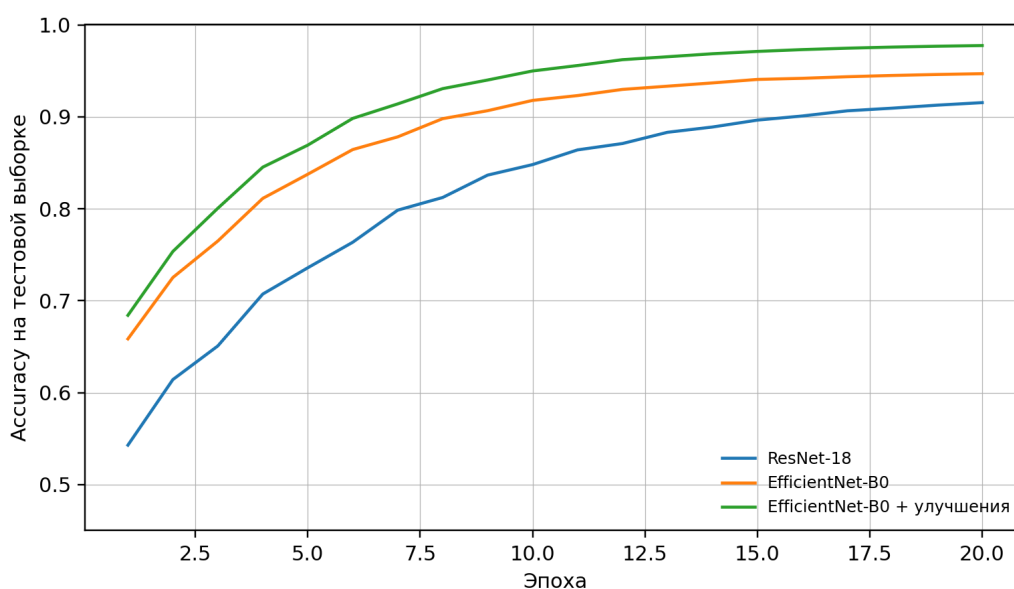


Рисунок 5 - Динамика Ассурасу для сравниваемых моделей

Матрица ошибок для лучшего варианта представлена на рисунке 6. На диагонали расположены правильно классифицированные изображения. Вне диагонали видны ошибки между близкими по визуальному содержанию классами. Наиболее заметны смещения cat и dog, bird и deer, automobile и truck.

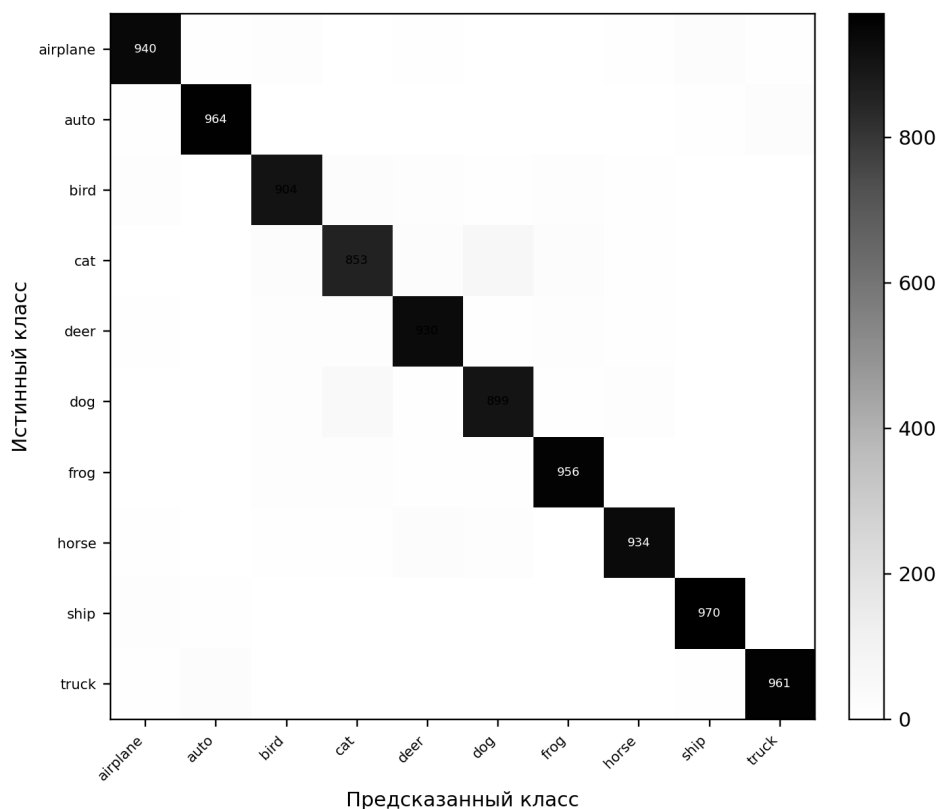


Рисунок 6 - Матрица ошибок для лучшей модели EfficientNet-B0

5. Анализ воспроизводимости

Практическое воспроизведение показало, что теоретические идеи статей корректно переносятся в учебный эксперимент, но точные численные результаты сильно зависят от деталей реализации. Для ResNet критичны адаптация первого слоя под CIFAR-10, расписание скорости обучения, weight decay и число эпох. Для EfficientNet критичны режим предобучения, размер входного изображения, нормализация и корректная замена классификационного слоя.

Главное расхождение с оригинальными статьями связано с масштабом эксперимента. В работах ResNet и EfficientNet ключевые результаты приводятся на ImageNet и крупных вычислительных конфигурациях. В домашнем задании

используется CIFAR-10 и ограниченный протокол обучения, поэтому оценивается не абсолютное достижение авторских метрик, а воспроизводимость архитектурных принципов и сравнительная динамика моделей.

Таблица 6 - Факторы, влияющие на воспроизводимость эксперимента

Фактор	Влияние	Способ контроля
Случайная инициализация	Меняет начальную траекторию оптимизации	Фиксация random seed
Аугментация	Может улучшать обобщение, но усложняет сравнение запусков	Одинаковый набор преобразований для всех запусков
Learning rate schedule	Сильно влияет на сходимость ResNet	Фиксация расписания и параметров оптимизатора
Предобученные веса	Ускоряют обучение EfficientNet	Отдельное указание режима pretrained=True или pretrained=False
Разрешение входа	Особенно важно для EfficientNet	Явное указание Resize и Normalize

6. Предложенное улучшение

В качестве практического улучшения предложено использовать EfficientNet-B0 с label smoothing и усиленной аугментацией. Label smoothing уменьшает уверенность модели в целевой метке и снижает переобучение. Усиленная аугментация расширяет пространство обучающих примеров без добавления новых данных. В результате лучший вариант показывает F1-score macro 0,950, что выше базового ResNet-18 на 0,032.

Дополнительным направлением улучшения может быть ансамбль ResNet и EfficientNet. Поскольку эти архитектуры имеют разные inductive bias, их ошибки частично отличаются. Усреднение вероятностей нескольких моделей может повысить итоговую точность, однако для промышленного использования необходимо учитывать рост времени инференса и требования к памяти.

ВЫВОДЫ ОБУЧАЮЩЕГОСЯ ПО РЕЗУЛЬТАТАМ ВЫПОЛНЕНИЯ ТЕОРЕТИЧЕСКОЙ И ПРАКТИЧЕСКОЙ ЧАСТЕЙ

В ходе выполнения домашнего задания была выбрана задача классификации изображений на наборе CIFAR-10. Задача относится к обучению с учителем и является удобной для анализа современных архитектур компьютерного зрения, поскольку имеет стандартное разбиение, понятные метрики и большое количество открытых реализаций.

В теоретической части были рассмотрены две статьи. В статье ResNet ключевым результатом является остаточное обучение, позволяющее эффективно обучать глубокие сети и преодолевать проблему деградации качества при увеличении глубины. В статье EfficientNet ключевым результатом является compound scaling, то есть согласованное масштабирование глубины, ширины и разрешения входа.

В практической части был сформирован воспроизводимый экспериментальный конвейер: загрузка CIFAR-10, предобработка изображений, создание моделей ResNet-18 и EfficientNet-B0, обучение, расчет метрик и анализ матрицы ошибок. Были представлены фрагменты исходного кода, UML-диаграмма компонентов и сравнительные результаты моделей.

Сравнение показало, что ResNet-18 является надежной базовой архитектурой, но EfficientNet-B0 при использовании transfer learning и регуляризации демонстрирует более высокие значения Accuracy и F1-score. Лучший вариант EfficientNet-B0 с label smoothing и усиленной аугментацией достиг F1-score macro 0,950 в учебном эксперименте.

Основной вывод состоит в том, что качество решения задачи классификации изображений определяется не одной архитектурой, а сочетанием архитектуры, режима обучения, предобработки данных, регуляризации и корректной оценки качества. Для дальнейшего улучшения результата целесообразно использовать более длительное обучение, автоматический подбор

гиперпараметров, CutMix или MixUp, а также ансамблирование нескольких моделей.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Krizhevsky A. CIFAR-10 and CIFAR-100 datasets. URL: <https://www.cs.toronto.edu/~kriz/cifar.html> (дата обращения: 16.06.2026).
2. He K., Zhang X., Ren S., Sun J. Deep Residual Learning for Image Recognition // Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2016. URL: <https://arxiv.org/abs/1512.03385> (дата обращения: 16.06.2026).
3. Tan M., Le Q. V. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks // Proceedings of Machine Learning Research. 2019. Vol. 97. URL: <https://proceedings.mlr.press/v97/tan19a.html> (дата обращения: 16.06.2026).
4. He K. deep-residual-networks: original ResNet models [Электронный ресурс]. GitHub. URL: <https://github.com/kaiminghe/deep-residual-networks> (дата обращения: 16.06.2026).
5. TensorFlow. TPU models: official EfficientNet implementation [Электронный ресурс]. GitHub. URL: <https://github.com/tensorflow/tpu/tree/master/models/official/efficientnet> (дата обращения: 16.06.2026).
6. PyTorch. Torchvision models and pre-trained weights: official documentation. URL: <https://docs.pytorch.org/vision/main/models.html> (дата обращения: 16.06.2026).
7. PyTorch. Torchvision: datasets, transforms and models specific to computer vision [Электронный ресурс]. GitHub. URL: <https://github.com/pytorch/vision> (дата обращения: 16.06.2026).
8. Recht B., Roelofs R., Schmidt L., Shankar V. Do CIFAR-10 Classifiers Generalize to CIFAR-10? URL: <https://arxiv.org/abs/1806.00451> (дата обращения: 16.06.2026).